WILLIAM E. BOEING DEPARTMENT OF
AERONAUTICS & ASTRONAUTICS
College of Engineering

AA411
Fault Detector Checkout Test Procedure

Fault Detector Team

Part Name: Fault Detector Code

Part Number: DGFDN1

Date: 2026 / 05 / 13
yyyy mm dd

Results: _____

Test Team:

Name	**Initials**
Joshua Rolfe	JR
Mak Sukimoto	MS
Dante Weerasooriya	DW

Test Objective

The purpose of this test campaign is to verify through testing that the Fault Detector subsystem satisfies all requirements defined for the subsystem, MSR-1 and MSR-2, and performs reliably across nominal, faulted, false-fault, and synthetic windowing scenarios. The Fault Detector is a subsystem of the Diagnoser and is responsible for determining whether Mission (MI) data deviates from Digital Twin (DT) predictions in a way that indicates a sustained system fault. The experiment evaluates the subsystem's behavior by running missioncompare_version_4 on a structured set of datasets and synthetic cases. The objective is to confirm that the subsystem correctly trims mission data, computes percent differences, applies window-based scoring, and identifies sustained deviations while rejecting transient or environmentally induced anomalies.

This test campaign verifies Requirement MSR-1 by confirming that the subsystem trims the first and last 20% of every dive profile before performing any comparison. This behavior is validated through pre-tests that ensure trimming only occurs when inputs are valid, windowing tests that confirm early- and late-mission spikes are removed, and synthetic tests that confirm trimming does not distort window scoring. Measurements used to validate MSR-1 include the number of samples removed, the time indices of trimmed segments, zero-removal counts, and the absence of failures when deviations occur outside the trimmed region.

The campaign verifies Requirement MSR-2 by confirming that the subsystem correctly computes percent differences between trimmed DT and MI datasets and flags variables whose windowed percent difference exceeds 40%. This behavior is validated using nominal dives (no failures expected), known-fault dives (failures expected), false-fault dives (environmental anomalies), and synthetic concentrated-spike tests. Measurements used to validate MSR-2 include percent-difference arrays, window scores, maximum window score per variable, excursion intervals, and the contents of Diagnose.Failures.

Across all test cases, the following data products are collected and archived: percent-difference arrays, windowed scores, failure lists, excursion intervals, zero-removal counts, generated plots, and console output. These measurements are used to verify trimming behavior, window scoring, and failure detection accuracy.

At the mission level, this test campaign validates that the Fault Detector can correctly identify nominal dives, correctly flag dives with rudder or wing faults, avoid false positives on environmentally anomalous dives, and consistently apply trimming and windowing logic across all datasets. Successful completion of this test campaign demonstrates that the subsystem is ready for integration into the full Diagnoser pipeline.

Equipment Required

Qty	Description	Specs/Calibration	Check
1	Seaglider Piloting Laptop or workstation with MATLAB	Must have MATLAB R2024b or later installed; must be able to run .m scripts and load .eng files; no calibration required	
1	.eng mission data files (Edmonds Dives 5, 6, 8, 11)	Files must be complete and uncorrupted; verify timestamps and file sizes match expected values; no login required	
1	missioncompare_version_4.m (Fault Detector script)	Must be the correct version; verify timestamp or Git hash; ensure no local modifications	
1	missioncompare_unpack_eng_to_dive_data.m (ENG-to-cell-array converter)	Must correctly convert .eng files into MI_Output cell arrays; verify variable names preserved	
1	Directory for MS_Comparator_Plots/	MATLAB must have permission to create folders and save .png files; verify write access	

Test Procedure:

1. Pre-Test Setup and Data Integrity Checks

1.1 Ensure that the Seaglider Piloting Laptop or workstation is available.

1.2 Confirm that the Seaglider Piloting Laptop or workstation has MATLAB installed.

OK? _____

1.3 Open MATLAB. Confirm MATLAB version using the ver command.

Measured version: _____

1.4 Verify that the measured MATLAB version meets the requirement (R2024b or later)?

OK? _____

1.5 Navigate to the MATLAB working directory where the Fault Detector scripts are stored.

Directory Path: _____

1.6 Confirm the presence of the required scripts in the working directory:

- missioncompare_version_4.m
- missioncompare_unpack_eng_to_dive_data.m

OK? _____

1.7 Confirm that all required .eng data files are present in the data folder:

- Edmonds Dive 5
- Edmonds Dive 6
- Edmonds Dive 8
- Edmonds Dive 11

Data folder path: _____

OK? _____

1.8 Open each .eng file to confirm it contains data (not empty or corrupted). You can check by opening it in a text editor or by running the unpack script and verifying non-empty output.

- File checked #1: _____ OK? _____
- File checked #2: _____ OK? _____
- File checked #3: _____ OK? _____
- File checked #4: _____ OK? _____
- File checked #5: _____ OK? _____

1.9 Confirm that MATLAB has permission to create and write to the MS_Comparator_Plots/ directory. Test by creating a temporary file in the directory.

OK? _____

1.10 Ensure that the laptop has sufficient free disk space for plots and .mat files.

Required: ≥ 1 GB free

Measured free space: _____ GB

OK? _____

2. Dataset Creation and Format Verification

2.1 Record all hardcoded parameters used by the Fault Detector. Open missioncompare_version_4.m and locate each parameter. Record the values exactly as they appear in the code.

Window size:
Expected: 20 samples

Measured (windowSize): _____

OK? _____

Failure threshold:

Expected: Window score > 40

Measured (failureThreshold): _____
OK? _____

Default tolerance
Expected: 2.0%

Measured (defaultTolerance): _____
OK? _____

Pitch / Roll / Heading tolerance
Expected: 6.6%

Measured (toleranceMap): _____
OK? _____

Pitch / Roll / Heading rate tolerance
Expected: 3.3%

Measured (toleranceMap): _____
OK? _____

Trimming rule
Expected: Middle 60% (drop first & last 20%)

Measured (Section 9.2): _____
OK? _____

Zero-removal rule
Expected: DT or MI = 0 → dropped

Measured (Section 9.1): _____
OK? _____

Diff metric
Expected: MAX window score

Measured (Section 9.9): _____
OK? _____

2.2 Convert Edmonds Dive 5 .eng file into MI_Output

2.2.1 Run the ENG-to-cell-array converter.
Command: MI_Output = missioncompare_unpack_eng_to_dive_data('Edmonds_Dive5.eng');
OK? _____

2.2.2 Confirm MI_Output exists and is not empty.
Check: whos MI_Output

OK? _____

2.2.3 Confirm MI_Output has the correct structure.
Row 1: variable name strings
Column 1: numeric time vector
Rows 2:end, Columns 2:end: numeric data

OK? _____

2.3 Convert Edmonds Dive 6 .eng file into MI_Output

2.3.1 Run the ENG-to-cell-array converter.
Command: MI_Output = missioncompare_unpack_eng_to_dive_data('Edmonds_Dive6.eng');
OK? _____

2.3.2 Confirm MI_Output exists and is not empty.
Check: whos MI_Output

OK? _____

2.3.3 Confirm MI_Output has the correct structure.
Row 1: variable name strings
Column 1: numeric time vector
Rows 2:end, Columns 2:end: numeric data

OK? _____

2.4 Convert Edmonds Dive 8 .eng file into MI_Output

2.4.1 Run the ENG-to-cell-array converter.
Command: MI_Output = missioncompare_unpack_eng_to_dive_data('Edmonds_Dive8.eng');
OK? _____

2.4.2 Confirm MI_Output exists and is not empty.
Check: whos MI_Output

OK? _____

2.4.3 Confirm MI_Output has the correct structure.
Row 1: variable name strings
Column 1: numeric time vector
Rows 2:end, Columns 2:end: numeric data

OK? _____

2.5 Convert Edmonds Dive 11 .eng file into MI_Output

2.5.1 Run the ENG-to-cell-array converter.
Command: MI_Output = missioncompare_unpack_eng_to_dive_data('Edmonds_Dive11.eng');
OK? _____

2.5.2 Confirm MI_Output exists and is not empty.
Check: whos MI_Output

OK? _____

2.5.3 Confirm MI_Output has the correct structure.
Row 1: variable name strings
Column 1: numeric time vector
Rows 2:end, Columns 2:end: numeric data

OK? _____

2.6 Confirm all MI_Output datasets are ready for use

2.6.1 Final readiness check

- All .eng files converted
- All MI_Output arrays exist
- All MI_Output arrays have correct structure
- No empty arrays
- No missing variables

Ready to proceed?
OK? _____

3. Pre-Test Dataset Construction and Synthetic Case Generation

3.1 Create and name MI_Output datasets for all mission files. Run the unpacking function for each .eng file and assign the output to the correct variable name.

3.1.1 Create MI_Output_Edmonds5
Command:
MI_Output_Edmonds5 = missioncompare_unpack_eng_to_dive_data('Edmonds_Dive5.eng');

OK? _____

3.1.2 Confirm MI_Output_Edmonds5 exists and contains data
Check: whos MI_Output_Edmonds5

OK? _____

3.1.3 Create MI_Output_Edmonds6
Command:
MI_Output_Edmonds6 = missioncompare_unpack_eng_to_dive_data('Edmonds_Dive6.eng');

OK? _____

3.1.4 Confirm MI_Output_Edmonds6 exists and contains data
Check: whos MI_Output_Edmonds6

OK? _____

3.1.5 Create MI_Output_Edmonds8
Command:
MI_Output_Edmonds8 = missioncompare_unpack_eng_to_dive_data('Edmonds_Dive8.eng');
OK? _____

3.1.6 Confirm MI_Output_Edmonds8 exists and contains data
Check: whos MI_Output_Edmonds8
OK? _____

3.1.7 Create MI_Output_Edmonds11
Command:
MI_Output_Edmonds11 = missioncompare_unpack_eng_to_dive_data('Edmonds_Dive11.eng');
OK? _____

3.1.8 Confirm MI_Output_Edmonds11 exists and contains data
Check: whos MI_Output_Edmonds11
OK? _____

3.2 Set the DT_Output dataset for all pre-tests

3.2.1 Assign DT_Output from MI_Output_Edmonds6
Command:
DT_Output = MI_Output_Edmonds6;
OK? _____

3.2.2 Confirm DT_Output exists and contains valid data
Check: whos DT_Output
OK? _____

3.2.3 Confirm DT_Output has correct structure
Row 1: variable name strings
Column 1: numeric time vector
Rows 2:end, Columns 2:end: numeric data
OK? _____

3.3 TEST-PRE-01: Column count mismatch

3.3.1 Create MI_Output_PRE01
MI_Output_PRE01 = DT_Output;
OK? _____

3.3.2 Add one extra column

MI_Output_PRE01{1,end+1} = 'ExtraVar';
MI_Output_PRE01(2:end,end+1) = {0};

OK? _____

3.3.3 Run the Fault Detector
Command:
Diagnose_PRE01 = missioncompare_version_4(DT_Output, MI_Output_PRE01);
Output/Error Message: _____

3.3.4 Compare to expected behavior
Expected Error ID: MissionCompare:VariableCountMismatch

Matches expected? YES / NO

3.3.5 Pass Criteria
Error thrown, no partial output.

Pass? _____

3.4 TEST-PRE-02: Variable name mismatch

3.4.1 Create MI_Output_PRE02
MI_Output_PRE02 = DT_Output;

OK? _____

3.4.2 Modify one variable name
MI_Output_PRE02{1,2} = 'WrongName';

OK? _____

3.4.3 Run the Fault Detector
Diagnose_PRE02 = missioncompare_version_4(DT_Output, MI_Output_PRE02);
Output/Error Message: _____

3.4.4 Compare to expected behavior
Expected Error ID: MissionCompare:VariableNameMismatch

Matches expected? YES / NO

3.4.5 Pass Criteria
Error thrown, no partial output.

Pass? _____

3.5 TEST-PRE-03: Time vector length mismatch

3.5.1 Create MI_Output_PRE03
MI_Output_PRE03 = DT_Output;

OK? _____

3.5.2 Remove one row
MI_Output_PRE03(2:end,:) = MI_Output_PRE03(2:end-1,:);

OK? _____

3.5.3 Run the Fault Detector
Diagnose_PRE03 = missioncompare_version_4(DT_Output, MI_Output_PRE03);

Output/Error Message: _____

3.5.4 Compare to expected behavior
Expected Error ID: MissionCompare:TimeMismatch

Matches expected? YES / NO

3.5.5 Pass Criteria
Error thrown, no partial output.

Pass? _____

3.6 TEST-PRE-04: All-zero variable column

3.6.1 Create MI_Output_PRE04
MI_Output_PRE04 = DT_Output;

OK? _____

3.6.2 Set one variable column to all zeros in both DT and MI

Variable selected: _____

Example:
DT_Output(2:end,3) = {0};
MI_Output_PRE04(2:end,3) = {0};

OK? _____

3.6.3 Run the Fault Detector
Diagnose_PRE04 = missioncompare_version_4(DT_Output, MI_Output_PRE04);

Output Summary: _____

3.6.4 Compare to expected behavior
Expected:

- Zeros removed
- Diff = NaN
- No crash
- Other variables processed normally

Matches expected? YES / NO

3.6.5 Pass Criteria
All expected behaviors observed.

Pass? _____

3.7 TEST-PRE-05: Plot folder creation

3.7.1 Ensure MS_Comparator_Plots/ does NOT exist
Delete or rename if needed.

OK? _____

3.7.2 Run a valid test case such as the following:
Diagnose_PRE05 = missioncompare_version_4(DT_Output, MI_Output_Edmonds5);

OK? _____

3.7.3 Compare to expected behavior
Expected:

- Folder created
- .png files saved
- Diagnose.Plots.varName contains valid paths

Matches expected? YES / NO

3.7.4 Pass Criteria
Folder created and plots saved.

Pass? _____

3.8 TEST-WIN-01: Early-mission spike (should be trimmed away)

3.8.1 Create MI_WIN01 from DT_Output
Command: MI_WIN01 = DT_Output;

OK? _____

3.8.2 Inject a large spike in the first 20% of samples

Variable selected: _____

Example command:
MI_WIN01(2:round(0.2*end),3) = {DT_Output{2,3} * 5};

OK? _____

3.8.3 Run the Fault Detector
Command: Diagnose_WIN01 = missioncompare_version_4(DT_Output, MI_WIN01);

Output Summary: _____

3.8.4 Compare to expected behavior
Expected:

- Spike occurs entirely in first 20%
- Trimming removes first 20%
- No failures should be flagged

Matches expected? YES / NO

3.8.5 Pass Criteria
Diagnose.Failures must be empty.

Pass? _____

3.9 TEST-WIN-02: Late-mission spike (should be trimmed away)

3.9.1 Create MI_WIN02 from DT_Output
MI_WIN02 = DT_Output;

OK? _____

3.9.2 Inject a spike in the last 20% of samples

Variable selected: _____

Example:
MI_WIN02(round(0.8*end):end,3) = {DT_Output{2,3} * 5};

OK? _____

3.9.3 Run the Fault Detector
Diagnose_WIN02 = missioncompare_version_4(DT_Output, MI_WIN02);

Output Summary: _____

3.9.4 Compare to expected behavior
Expected:

- Spike occurs entirely in last 20%
- Trimming removes last 20%
- No failures should be flagged

Matches expected? YES / NO

3.9.5 Pass Criteria
Diagnose.Failures must be empty.

Pass? _____

3.10 TEST-WIN-03: Sustained small difference (should NOT accumulate)
3.10.1 Create MI_WIN03 from DT_Output
MI_WIN03 = DT_Output;

OK? _____

3.10.2 Add a sustained 2.5% difference to all samples

Variable selected: _____

Example:
MI_WIN03(2:end,3) = {DT_Output{2,3} * 1.025};

OK? _____

3.10.3 Run the Fault Detector
Diagnose_WIN03 = missioncompare_version_4(DT_Output, MI_WIN03);

Output Summary: _____

3.10.4 Compare to expected behavior
Expected:

- Each sample is 2.5% above DT
- Tolerance = 2.0% → 0.5% over
- Window score = 0.5 × 20 = 10
- Threshold = 40 → no failure

Matches expected? YES / NO

3.10.5 Pass Criteria
Diagnose.Failures must be empty.

Pass? _____

3.11 TEST-WIN-04: Concentrated spike in a middle window

3.11.1 Create MI_WIN04 from DT_Output
MI_WIN04 = DT_Output;

OK? _____

3.11.2 Inject a 100% spike into a middle window
Choose a window that is not the first or last trimmed window.
Example:

- Total samples after trimming ≈ 60% of original
- Window size = 20
- Middle window = window #2

Window start index: _____

Example command:
MI_WIN04(start:start+19,3) = {DT_Output{2,3} * 2};

OK? _____

3.11.3 Run the Fault Detector
Diagnose_WIN04 = missioncompare_version_4(DT_Output, MI_WIN04);

Output Summary: _____

3.11.4 Expected behavior
Expected:

- Diagnose.Failures empty
- Excursion not propagated
- No false fault

Matches expected? YES / NO

3.11.5 Pass Criteria
Diagnose.Failures must be empty.

Pass? _____

4. Main Test Procedure

4.1 Mission Test 1: Edmonds Dive 6 vs DT (Nominal Self-Comparison)

4.1.1 Assign DT and MI datasets
DT_Output = MI_Output_Edmonds6;
MI_Output = MI_Output_Edmonds6;

OK? _____

4.1.2 Confirm datasets exist and contain data
Check: whos DT_Output
Check: whos MI_Output

OK? _____

4.1.3 Run the Fault Detector
Command: Diagnose_Edmonds6 = missioncompare_version_4(DT_Output, MI_Output);
Error/Warning Message: _____
OK? _____

4.1.4 Inspect and record all fields of Diagnose_Edmonds6

Variable Names
Diagnose_Edmonds6.VariableNames
Value: _____ OK? _____

Percentage Difference
Diagnose_Edmonds6.difference
Recorded? YES / NO OK? _____

Diff Metric
Diagnose_Edmonds6.Diff
Recorded? YES / NO OK? _____

Failures
Diagnose_Edmonds6.Failures
Empty? YES / NO
Value: _____ OK? _____

Plots
Diagnose_Edmonds6.Plots

Saved? YES / NO OK? _____

Excursions
Diagnose_Edmonds6.Excursions

Value: _____ OK? _____

Count Above Tolerance
Diagnose_Edmonds6.Count

Value: _____ OK? _____

Total Excess Difference
Diagnose_Edmonds6.Excess

Value: _____ OK? _____

Window Scores
Diagnose_Edmonds6.WindowScores

Value: _____ OK? _____

Removed Zeros
Diagnose_Edmonds6.RemovedZeros

Value: _____ OK? _____

4.1.5 Evaluate results
Expected: No failures

Matches expected? YES / NO
Pass? _____

Save file: Diagnose_Edmonds6.mat

OK? _____

4.2 Mission Test 2: Edmonds Dive 5 vs DT (Nominal Mission)

4.2.1 Assign DT and MI datasets
DT_Output = MI_Output_Edmonds6;
MI_Output = MI_Output_Edmonds5;

OK? _____

4.2.2 Confirm datasets exist
Check: whos DT_Output
Check: whos MI_Output

OK? _____

4.2.3 Run the Fault Detector

Diagnose_Edmonds5 = missioncompare_version_4(DT_Output, MI_Output);

Error/Warning Message: _____

OK? _____

4.2.4 Inspect and record all fields of Diagnose_Edmonds5

Variable Names

Diagnose_Edmonds5.VariableNames

Value: _____ OK? _____

Percentage Difference

Diagnose_Edmonds5.difference

Recorded? YES / NO OK? _____

Diff Metric

Diagnose_Edmonds5.Diff

Recorded? YES / NO OK? _____

Failures

Diagnose_Edmonds5.Failures

Empty? YES / NO

Value: _____ OK? _____

Plots

Diagnose_Edmonds5.Plots

Saved? YES / NO OK? _____

Excursions

Diagnose_Edmonds5.Excursions

Value: _____ OK? _____

Count Above Tolerance

Diagnose_Edmonds5.Count

Value: _____ OK? _____

Total Excess Difference

Diagnose_Edmonds5.Excess

Value: _____ OK? _____

Window Scores

Diagnose_Edmonds5.WindowScores

Value: _____ OK? _____

Removed Zeros
Diagnose_Edmonds5.RemovedZeros
Value: _____ OK? _____

4.2.5 Evaluate results
Expected: No failures

Matches expected? YES / NO
Pass? _____

Save file: Diagnose_Edmonds5.mat

OK? _____

4.3 Mission Test 3: Edmonds Dive 8 vs DT (Rudder Fault)

4.3.1 Assign DT and MI datasets
DT_Output = MI_Output_Edmonds6;
MI_Output = MI_Output_Edmonds8;

OK? _____

4.3.2 Confirm datasets exist
Check: whos DT_Output
Check: whos MI_Output

OK? _____

4.3.3 Run the Fault Detector
Diagnose_Edmonds8 = missioncompare_version_4(DT_Output, MI_Output);
Error/Warning Message: _____
OK? _____

4.3.4 Inspect and record all fields of Diagnose_Edmonds8

Variable Names
Diagnose_Edmonds8.VariableNames
Value: _____ OK? _____

Percentage Difference
Diagnose_Edmonds8.difference

Recorded? YES / NO OK? _____

Diff Metric
Diagnose_Edmonds8.Diff

Recorded? YES / NO OK? _____

Failures

Diagnose_Edmonds8.Failures

Empty? YES / NO

Value: _____ OK? _____

Plots

Diagnose_Edmonds8.Plots

Saved? YES / NO OK? _____

Excursions

Diagnose_Edmonds8.Excursions

Value: _____ OK? _____

Count Above Tolerance

Diagnose_Edmonds8.Count

Value: _____ OK? _____

Total Excess Difference

Diagnose_Edmonds8.Excess

Value: _____ OK? _____

Window Scores

Diagnose_Edmonds8.WindowScores

Value: _____ OK? _____

Removed Zeros

Diagnose_Edmonds8.RemovedZeros

Value: _____ OK? _____

4.3.5 Evaluate results

Expected: Failures present (rudder loss)

Matches expected? YES / NO

Pass? _____

Save file: Diagnose_Edmonds8.mat

OK? _____

4.4 Mission Test 4: Edmonds Dive 11 vs DT (Faulted Mission)

4.4.1 Assign DT and MI datasets

DT_Output = MI_Output_Edmonds6;

MI_Output = MI_Output_Edmonds11;

OK? _____

4.4.2 Confirm datasets exist

Check: whos DT_Output
Check: whos MI_Output

OK? _____

4.4.3 Run the Fault Detector
Diagnose_Edmonds11 = missioncompare_version_4(DT_Output, MI_Output);
Error/Warning Message: _____
OK? _____

4.4.4 Inspect and record all fields of Diagnose_Edmonds11

Variable Names
Diagnose_Edmonds11.VariableNames
Value: _____ OK? _____

Percentage Difference
Diagnose_Edmonds11.difference
Recorded? YES / NO OK? _____

Diff Metric
Diagnose_Edmonds11.Diff
Recorded? YES / NO OK? _____

Failures
Diagnose_Edmonds11.Failures
Empty? YES / NO
Value: _____ OK? _____

Plots
Diagnose_Edmonds11.Plots
Saved? YES / NO OK? _____

Excursions
Diagnose_Edmonds11.Excursions
Value: _____ OK? _____

Count Above Tolerance
Diagnose_Edmonds11.Count
Value: _____ OK? _____

Total Excess Difference
Diagnose_Edmonds11.Excess
Value: _____ OK? _____

Window Scores
Diagnose_Edmonds11.WindowScores
Value: _____ OK? _____

Removed Zeros
Diagnose_Edmonds11.RemovedZeros
Value: _____ OK? _____

4.4.5 Evaluate results
Expected: Failures present (wing loss)

Matches expected? YES / NO
Pass? _____

Save file: Diagnose_Edmonds11.mat

OK? _____

5. Shut down Procedure

5.1 Verify all required output files exist

5.1.1 Check for Diagnose files
In your MATLAB working directory, confirm that these four files are present:

- Diagnose_Edmonds6.mat
- Diagnose_Edmonds5.mat
- Diagnose_Edmonds8.mat
- Diagnose_Edmonds11.mat

All four files present?
OK? _____

5.2 Verify plot folder is complete

5.2.1 Open the plot directory
Navigate to:
MS_Comparator_Plots/

5.2.2 Confirm the following

- The folder exists
- It contains .png files for each mission
- No plots are missing
- Files open correctly
- Plot directory verified?

OK? _____

5.3 Archive all test results

5.3.1 Copy required files to long-term storage
Copy the following to your designated archive location:

- All Diagnose_*.mat files
- The entire MS_Comparator_Plots/ folder
- Your test log / notes

Archive completed successfully?
OK? _____

5.4 Clean the MATLAB environment

5.4.1 Close all open figures
Run in MATLAB:
close all

All figures closed?
OK? _____

5.4.2 Clear the workspace
Run:
clear all; clc

Workspace empty?
OK? _____

5.4.3 Close MATLAB
Exit MATLAB normally.

MATLAB closed?
OK? _____

5.5 Clean up the working directory

5.5.1 Remove temporary or intermediate files
Delete the following if they exist:

- Synthetic test datasets
- Temporary .mat files not part of final results
- Debug scripts or scratch files

5.5.2 Confirm only required files remain
The directory should now contain ONLY:

- Mission .eng files
- missioncompare_version_4.m
- missioncompare_unpack_eng_to_dive_data.m
- Diagnose_*.mat
- MS_Comparator_Plots/

Directory cleaned and verified?
OK? _____

5.6 Final documentation check

5.6.1 Confirm the test report is complete
Verify that:

- All mission sections (4.1–4.4) are filled out
- All Diagnose fields were recorded
- Pass/Fail decisions documented
- Notes on anomalies included
- Shutdown procedure completed

Documentation complete?
OK? _____

Change Log

Ver	Date	By	E-mail	Change
1.0	5/8/2026	Dante Weerasooriya	dantew1@uw.edu	Initial release.
2.0	5/10/2026	Dante Weerasooriya	dantew1@uw.edu	Updated procedure to include unpacking function for data files
3.0	5/13/2026	Dante Weerasooriya	dantew1@uw.edu	Updated tests to match the updated test matrix